



PROJECT REPORT

NEXVION

Project Title	NEXVION
Internship Domain	DEVOPS
Submitted By	ABRAHAM MAKUR DHUOR
Submission Date	08.10.2026

TABLE OF CONTENTS

No.	Section	Page No.
1	Project Overview	2-8
2	Problem Statement	8
3	Project Objectives	8-10
4	Project Scope	11-13
5	Technologies & Tools Used	14-17
6	System / Project Architecture	18-19
7	Project Workflow	20-24
8	Project Implementation	25-26
9	Key Features / Functionalities	26
10	Testing & Validation	27-29
11	Results & Outcomes	29
12	Project Screenshots & Evidence	30-34
13	Challenges & Solutions	35
14	Learning Outcomes	36
15	Conclusion	37
16	Future Enhancements	38
17	References	40

1. Project Overview

1.1 Project Description

Nexvion is a production-style e-commerce platform created to demonstrate modern DevOps and cloud deployment practices. It provides a static online shopping experience with a home page, product catalogue, search and filtering, shopping cart, user registration/login screens, checkout, and order confirmation.

The project repository is available at <https://github.com/dhuor2001-debug/Nexvion>.

What the project does

Nexvion allows users to:

- Browse a catalogue of products.
- Search, filter, sort, and view product information.
- Add products to a shopping cart.
- Adjust item quantities and remove products.
- Create a demo account and log in.
- Proceed through a payment and checkout flow.
- Select card, UPI, or cash-on-delivery demo options.
- Receive an order confirmation.
- Retain demo account, cart, and order data in browser localStorage.

The application is packaged as an unprivileged Nginx container. Nginx serves the static frontend files and exposes a `/healthz` endpoint so deployment systems can confirm that the application is running.

Practical use of the completed project

The project can be practically used as:

1. A DevOps portfolio project

It demonstrates the ability to design, containerize, secure, deploy, monitor, and troubleshoot a complete application.

2. A CI/CD reference implementation

The Jenkins pipeline shows how source validation, security scanning, image publishing, deployment, and post-deployment health checks can be automated.

3. An infrastructure-as-code example

The Terraform and Ansible configuration demonstrates repeatable AWS provisioning and server configuration instead of manual setup.

4. A deployment training environment

Developers can run the same application locally with Docker, deploy it to a single AWS server, or run it on Kubernetes to compare deployment models.

5. A release-strategy demonstration

The rolling update, blue/green, and canary manifests provide examples of how production teams reduce deployment risk and control traffic during releases.

6. A troubleshooting and operations exercise

The project contains real operational workflows for diagnosing SSH access, firewall and security-group rules, container health checks, Nginx upstream failures, image-pull problems, and Jenkins deployment issues. These are recorded in ``docs/troubleshooting.md``.

Important limitation

Nexvion is a demonstration storefront rather than a production commerce backend. Authentication and payment are implemented only in the browser using `localStorage`. The project must not be used with real passwords, card numbers, or payment credentials.

To become a real production e-commerce system, it would need:

- A backend API.
- Secure server-side authentication.
- A database.
- Server-side order management.
- A real payment gateway.
- Secure secret management.
- HTTPS and domain configuration.
- Business, privacy, and payment compliance controls.

In its completed form, Nexvion is most valuable as a practical demonstration of how a web application moves from source code to a tested, scanned, automated, cloud-deployed, and observable system.

1.2 Use Case

Nexvion represents the type of online storefront used by a small or medium-sized retailer that wants to publish a product catalogue and provide customers with a simple online purchasing experience. A business could use the platform to display products, allow customers to search and filter items, manage a shopping cart, and guide users through a checkout process.

The DevOps implementation models how the retailer's application can be delivered and operated reliably:

1. Developers commit changes to the source repository.
2. Jenkins validates the application and builds a Docker image.
3. Gitleaks and Trivy scan the source, image, and deployment configuration.
4. The approved image is published to Docker Hub.
5. The application is deployed to an AWS EC2 server or a Kubernetes cluster.
6. Health checks confirm that the new version is running.
7. Prometheus, Grafana, Filebeat, Logstash, and AI-assisted log analysis provide the foundation for monitoring and incident response.
8. Rolling, blue/green, or canary deployment patterns can be used to reduce release risk.

For example, a retailer may update product information or introduce a new page. Instead of manually copying files to a server, the team can commit the change and let the pipeline build, scan, publish, deploy, and verify the new version automatically.

Intended users and beneficiaries

Customers or visitors

Customers are the end users of the storefront. They can:

- Browse products.
- Search, filter, and sort the catalogue.
- Add products to a cart.
- Review quantities and totals.
- Complete a demonstration checkout flow.

- Receive an order confirmation.

In a real implementation, customers would also benefit from secure accounts, persistent orders, live inventory, and integrated payments.

Store owners and business operators

A small business owner or retailer could use the platform as the foundation for an online shop. The application gives the business:

- A digital product catalogue.
- A customer-facing storefront.
- A structure that can later be connected to inventory, order, and payment systems.
- A deployment process that reduces dependence on manual server administration.

The current project is a technical demonstration, so the browser-only authentication and payment features must be replaced before handling real customer data or transactions.

Developers

Developers use Nexvion to build and maintain storefront features. They benefit from:

- A clear static frontend structure.
- Reproducible local Docker development.
- Automated build and validation stages.
- Versioned container images.
- Health checks for confirming application availability.
- Documented deployment and troubleshooting procedures.

The project also provides a practical reference for implementing frontend changes without directly modifying production servers.

DevOps and platform engineers

DevOps engineers can use Nexvion to practice and demonstrate:

- Infrastructure provisioning with Terraform.

- Linux server configuration with Ansible.
- Container image creation and publishing.
- Jenkins CI/CD pipeline design.
- AWS deployment and network security.
- Kubernetes and Helm deployments.
- Rolling, blue/green, and canary release strategies.
- Security scanning with Gitleaks and Trivy.
- Metrics, logs, dashboards, and incident analysis.

This makes the project useful for training, technical assessments, portfolio demonstrations, and internal proof-of-concept work.

Security and operations teams

Security and operations teams can use the project to examine:

- Restricted SSH access through AWS security groups and UFW.
- SSH hardening and fail2ban.
- Non-root containers.
- Read-only container filesystems.
- Dropped Linux capabilities.
- Resource requests and limits.
- Secret handling through external Kubernetes secrets.
- Source and image vulnerability scanning.
- Application health and gateway failure troubleshooting.

These controls demonstrate how security and reliability can be incorporated into the delivery process rather than added only after deployment.

Students and educators

Nexvion is also suitable as a teaching project for students learning cloud computing, DevOps, or software deployment. It presents a complete path from frontend source code to:

- Docker packaging.
- Automated testing and security checks.

- Infrastructure creation.
- Cloud deployment.
- Kubernetes orchestration.
- Monitoring and logging.
- Troubleshooting real operational failures.

Overall benefit

The main beneficiaries are teams that need a realistic but manageable example of a modern application delivery platform. Nexvion connects the customer-facing storefront with the engineering systems required to build, secure, deploy, monitor, and maintain it.

It is not currently a complete commercial e-commerce solution because it does not include a production backend, database, secure authentication, or real payment processing. Its practical value is as a working reference platform and learning environment that can be extended into a production system by adding those services.

2. Problem Statement

2.1 Existing Problem

Small e-commerce applications are often deployed manually, inconsistently, and without proper security, monitoring, or reliable recovery processes. This can cause deployment errors, downtime, difficult troubleshooting, and slow delivery of updates.

Nexvion addresses this by providing a standardized, automated, and secure process for building, deploying, monitoring, and maintaining an online storefront.

2.2 Proposed Approach

Small e-commerce businesses need a reliable way to present products online while deploying and maintaining the application efficiently. Nexvion addresses this by providing a containerized storefront with automated CI/CD, security scanning, infrastructure provisioning, cloud deployment, health checks, monitoring, and troubleshooting support.

3. Project Objectives

3.1 Objectives

The project was intended to achieve the following measurable and practical objectives:

1. Deliver a working e-commerce storefront

Provide a functional web interface where users can browse products, search and filter the catalogue, manage a shopping cart, and complete a demonstration checkout flow.

2. Containerize the application

Package the storefront into a reproducible Docker image using Nginx, with the application running as a non-root user on port 8080.

3. Provide reliable health verification

Implement a `/healthz` endpoint and container health checks so automated systems can confirm whether the application is running correctly.

4. Automate continuous integration and delivery

Create a Jenkins pipeline that validates required files, builds the image, tests Nginx configuration, pushes versioned images, deploys the application, and verifies the public deployment.

5. Introduce automated security scanning

Use Gitleaks to detect exposed secrets and Trivy to scan container images and configuration for high and critical vulnerabilities before deployment.

6. Provision cloud infrastructure consistently

Use Terraform to create the AWS VPC, public subnet, routing, security group, EC2 instance, encrypted storage, SSH key configuration, and Elastic IP without relying on manual setup.

7. Automate server configuration

Use Ansible to install Docker and required packages, configure swap, enable UFW and fail2ban, harden SSH access, and deploy the production Compose service.

8. Apply practical security controls

Restrict SSH access to an approved administrator CIDR, disable SSH password and root login, run the application as an unprivileged user, use a read-only filesystem where supported, drop unnecessary capabilities, and avoid committing real secrets.

9. Support multiple deployment models

Provide local Docker Compose deployment, AWS EC2 deployment, direct Kubernetes manifests, and a Helm chart so the application can be run in different environments.

10. Demonstrate safer release strategies

Include rolling updates, blue/green deployment, and canary deployment examples to show how new versions can be released with reduced downtime and risk.

11. Establish monitoring and logging foundations

Provide Prometheus and Grafana configuration for metrics, along with Filebeat and Logstash configuration for log collection and processing.

12. Improve operational troubleshooting

Document practical procedures for diagnosing SSH timeouts, firewall and security-group problems, container health failures, Nginx upstream errors, image-pull issues, Terraform failures, and Jenkins deployment problems.

13. Make deployments reproducible

Ensure that a new environment can be created from version-controlled configuration and documented commands rather than undocumented manual steps.

14. Demonstrate end-to-end DevOps capability

Connect application development, version control, containerization, security validation, infrastructure automation, deployment, monitoring, and incident troubleshooting into one complete workflow.

15. Provide a foundation for future production development

Create a structured base that can later be extended with a real backend API, database, secure authentication, inventory management, persistent orders, and a production payment gateway.

4. Project Scope

4.1 Included Scope

The project includes the design, development, containerization, deployment, and documentation of the Nexvion e-commerce platform.

Application functions

- Static home page and product catalogue.
- Product search, filtering, sorting, and stock display.
- Shopping cart management and quantity controls.
- Demo user registration, login, and logout.
- Demo checkout and order confirmation.
- Card, UPI, and cash-on-delivery demonstration options.
- Browser localStorage for demo user, cart, checkout, and order data.
- Nginx /healthz endpoint for application health verification.

Technical modules

- HTML, CSS, and JavaScript frontend.
- Nginx web server and reverse-proxy configuration.
- Docker image and Docker Compose configuration.
- Terraform AWS infrastructure provisioning.
- Ansible server configuration and security hardening.
- Jenkins CI/CD pipeline.
- Docker Hub image publishing.
- Kubernetes manifests and Helm chart.
- Rolling, blue/green, and canary deployment examples.
- Prometheus and Grafana monitoring configuration.
- Filebeat and Logstash logging configuration.
- AI-assisted log analysis and incident classification.

- Security scanning using Gitleaks and Trivy.
- Project architecture, deployment, implementation, and troubleshooting documentation.

Activities

- Source validation and container image builds.
- Nginx configuration testing.
- Automated security scans.
- AWS infrastructure creation.
- EC2 server configuration.
- Local, AWS, and Kubernetes deployment.
- Health-check verification and operational troubleshooting.
- Documentation of setup, deployment, security, and support procedures.

4.2 Limitations

- The application is primarily a frontend demonstration and has no production backend API.
- Authentication is simulated in the browser using localStorage and is not secure for real users.
- Payment processing is simulated; no real payment gateway is connected.
- There is no production database, inventory system, order-management system, or persistent server-side customer data.
- Product, cart, account, and order data are not shared between browsers or devices.
- HTTPS, domain management, TLS certificate automation, and a production DNS setup are not included.
- AWS deployment depends on valid AWS credentials, an available SSH key, correct administrator CIDR configuration, and accessible cloud resources.
- Jenkins deployment uses configured credentials and a fixed AWS host value that must be updated when infrastructure changes.
- Monitoring and logging configurations are provided as foundations and may require additional deployed services and storage.
- Kubernetes blue/green and canary configurations are demonstrations and require a functioning cluster and ingress controller.
- No formal load testing, disaster recovery implementation, backup restoration testing, or high-availability production architecture is included.
- The project does not include business processes such as tax calculation, shipping integration, refunds, returns, fraud detection, or regulatory compliance.
- External services such as Docker Hub, Google Fonts, and Unsplash images may be unavailable or subject to their own limits.

- The project is not intended to process real passwords, card numbers, or payment credentials without substantial additional security and backend development.



5. Technologies & Tools Used

5.1 Technology Stack

Category	Technologies
Frontend languages	HTML5, CSS3, JavaScript
Web server	Nginx 1.27 Alpine, Nginx Unprivileged
Containerization	Docker, Docker Compose v2
Infrastructure as Code	Terraform, AWS Provider
Cloud platform	Amazon Web Services, including VPC, EC2, Elastic IP, Internet Gateway, security groups, and encrypted EBS storage
Server configuration	Ansible, Ubuntu Server 24.04
CI/CD	Jenkins, Jenkins Pipeline, Shell scripting
Image registry	Docker Hub
Orchestration	Kubernetes
Kubernetes packaging	Helm
Kubernetes ingress	NGINX Ingress Controller
Monitoring	Prometheus, Grafana, kube-state-metrics, Node Exporter
Logging	Filebeat, Logstash, Nginx access and error logs
Security tools	Gitleaks, Trivy, UFW, fail2ban
Application analysis	Python and JSON-based incident reports
Data storage	Browser localStorage for demonstration data
Version control	Git and GitHub

Category	Technologies
External resources	Google Fonts and Unsplash images
Documentation	Markdown and draw.io

5.2 Purpose of Each Tool

Tool or technology	Purpose
HTML5	Defines the structure of the storefront pages, product catalogue, checkout page, and order confirmation interface.
CSS3	Provides page layout, styling, responsive presentation, product cards, forms, cart views, and checkout design.
JavaScript	Implements product filtering, search, sorting, cart management, demo authentication, checkout validation, and localStorage persistence.
Nginx	Serves static frontend files efficiently, provides the /healthz endpoint, adds response headers, and acts as a reverse proxy in local Compose.
Docker	Packages the application and its runtime into a consistent, portable container image.
Docker Compose	Runs the application locally and deploys the production service on the AWS EC2 host.
Terraform	Creates repeatable AWS infrastructure, including networking, EC2, security groups, SSH access, encrypted storage, and the Elastic IP.
AWS VPC	Provides an isolated virtual network for the cloud deployment.
AWS EC2	Hosts the production Docker Compose application on an Ubuntu server.
AWS Elastic IP	Provides a stable public address for the deployed application.
AWS Security Group	Controls inbound and outbound instance traffic, allowing public HTTP and restricted SSH access.
AWS EBS encryption	Protects the EC2 root volume at rest.

Tool or technology	Purpose
Ansible	Automates server setup, Docker installation, firewall configuration, SSH hardening, swap creation, and application deployment.
Ubuntu 24.04	Provides the operating-system environment for the AWS host.
Jenkins	Automates the CI/CD workflow from source validation through build, security scan, image publishing, deployment, and health verification.
Docker Hub	Stores and distributes versioned Nexvion container images.
Gitleaks	Detects accidentally committed credentials and secrets in the source repository.
Trivy	Scans container images and infrastructure configuration for vulnerabilities and security issues.
UFW	Provides host-level firewall rules for SSH and HTTP access.
fail2ban	Helps protect SSH and other services against repeated unauthorized login attempts.
Kubernetes	Provides container orchestration, replica management, rolling updates, service discovery, health probes, and resource controls.
Helm	Packages Kubernetes resources into a reusable and configurable deployment chart.
NGINX Ingress	Routes external HTTP traffic to the Kubernetes service.
Prometheus	Collects metrics from the Kubernetes and application environment.
Grafana	Visualizes metrics through dashboards for operational monitoring.
Filebeat	Collects and forwards server and container logs.
Logstash	Processes and routes collected log data for analysis and storage.
Python	Supports the AI-assisted log-analysis utility and incident classification workflow.
localStorage	Stores demonstration users, cart items, checkout data, and order information in the browser.
Git	Tracks source code, infrastructure, pipeline, and documentation changes.
GitHub	Hosts the project repository and provides collaborative version control.

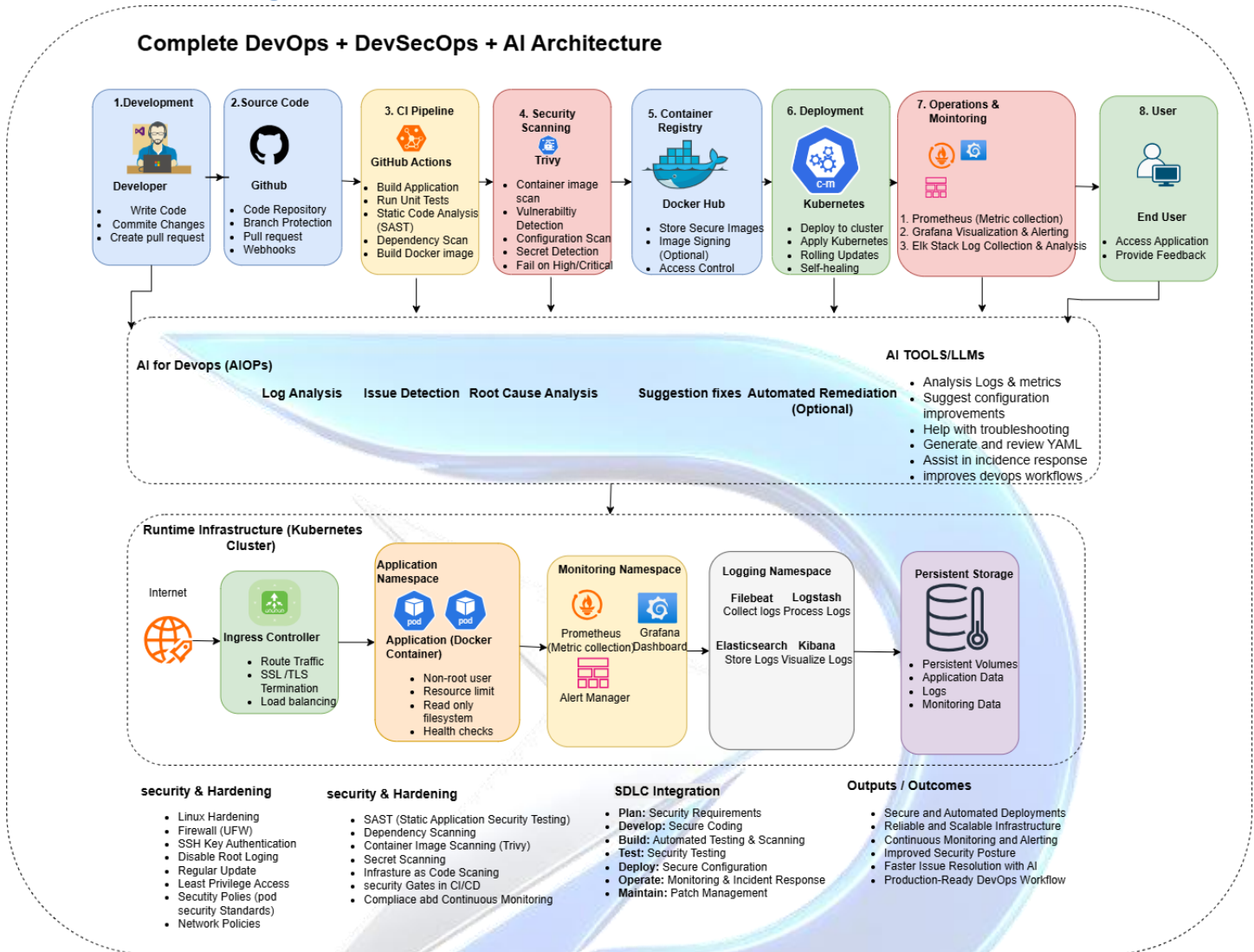
Tool or technology	Purpose
Mermaid	Represents the system architecture and deployment relationships as diagrams in Markdown documentation.

Together, these technologies provide a complete development and operations workflow: frontend development, container packaging, automated security validation, cloud infrastructure provisioning, deployment automation, orchestration, monitoring, logging, and troubleshooting.



6. System / Project Architecture

6.1 Architecture Diagram



6.2 Component Description

- **Docker:** Packages the application into a reproducible container image.
- **Docker Compose:** Supports local development and single-server deployment.
- **Terraform:** Provisions an AWS VPC, public subnet, route table, Internet Gateway, EC2 instance, security group, SSH key pair, encrypted storage, and Elastic IP.
- **Ansible:** Configures the Ubuntu server, installs Docker, enables UFW and fail2ban, configures SSH hardening, creates swap space, and deploys the production Compose service.
- **Jenkins:** Automates validation, image builds, Nginx configuration testing, security scanning, Docker Hub publishing, AWS deployment, and health verification.
- **Gitleaks and Trivy:** Scan source code, container images, and configuration for security problems.

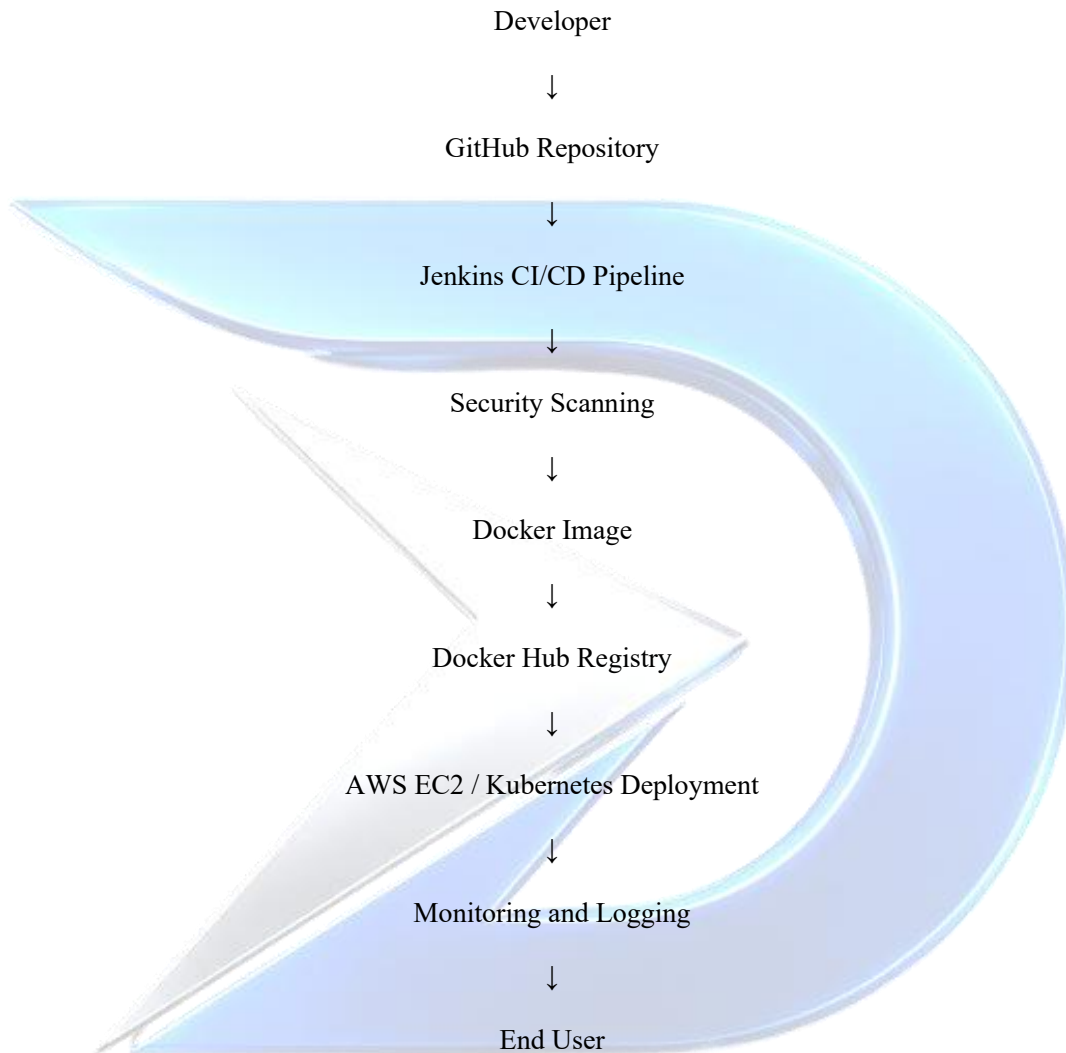
- **Kubernetes:** Provides deployment, service, ingress, health probes, resource limits, and security contexts.
- **Helm:** Provides a reusable and configurable Kubernetes chart.
- **Blue/green and canary examples:** Demonstrate alternative release strategies for safer application rollouts.
- **Prometheus and Grafana:** Provide a foundation for metrics and dashboard monitoring.
- **Filebeat and Logstash:** Provide a foundation for centralized log collection and processing.
- **AI-assisted log analysis:** Helps classify incidents and identify possible upstream or gateway failures.



7. Project Workflow

7.1 End-to-End Workflow

Sequence of Major Steps



1. Development input

The process begins when a developer creates or updates Nexvion source code, including:

- HTML pages
- CSS styles
- JavaScript functionality

- Docker configuration
- Terraform and Ansible files
- Kubernetes or Helm manifests

The developer tests the application locally using Docker Compose.

2. Source code submission

The changes are committed and pushed to the GitHub repository. GitHub stores:

- Application source code
- Infrastructure configuration
- Deployment manifests
- Jenkins pipeline definition
- Documentation

The repository acts as the central source of truth for the project.

3. CI/CD pipeline execution

A Jenkins pipeline detects the change and starts the delivery process. It:

1. Checks out the latest source code.
2. Verifies that required files exist.
3. Builds the Docker image.
4. Validates the Nginx configuration.
5. Runs the application container.
6. Performs health verification.

The project uses Jenkins for CI/CD rather than GitHub Actions shown in the supplied diagram.

4. Security scanning

Before deployment, the pipeline performs security checks:

- **Gitleaks** checks for accidentally committed secrets.
- **Trivy** scans the Docker image for vulnerabilities.
- **Trivy** scans Docker, Kubernetes, and Helm configuration.
- Critical security findings can fail the pipeline.

Only an image that passes the required checks should continue to deployment.

5. Container image creation

The application is packaged into a Docker image using Nginx. The image contains:

- Static HTML, CSS, and JavaScript files
- Nginx configuration
- /healthz health endpoint
- Container health check
- Non-root runtime configuration

The image listens internally on port 8080.

6. Container registry publishing

After successful validation and scanning, Jenkins pushes the image to Docker Hub.

Two tags are published:

- A build-specific tag, such as nexvion:25
- The latest tag

The build-specific tag provides traceability and makes rollback possible.

7. Infrastructure and deployment

The image can be deployed through multiple paths.

AWS EC2 deployment

Terraform creates the cloud infrastructure:

- VPC
- Public subnet
- Internet Gateway
- Route table
- Security group
- EC2 instance
- Encrypted storage
- Elastic IP

Ansible then configures the Ubuntu server by installing Docker, enabling UFW and fail2ban, hardening SSH, and deploying Docker Compose.

Jenkins connects to the EC2 server and runs:

docker compose up -d --pull always

The application is exposed publicly on port 80.

Kubernetes deployment

Alternatively, Kubernetes runs the image using:

- Deployment replicas
- ClusterIP Service
- NGINX Ingress
- Readiness and liveness probes
- Resource requests and limits
- Non-root security settings
- External Kubernetes secrets

The Helm chart provides a reusable and configurable version of the Kubernetes deployment.

8. Release strategies

The project demonstrates several deployment strategies:

- Rolling update: Replaces old pods gradually.
- Blue/green deployment: Runs two versions and switches traffic between them.
- Canary deployment: Sends a controlled percentage of traffic to the new version before full release.

These strategies reduce downtime and deployment risk.

9. Operations, monitoring, and logging

After deployment, the system is monitored and operated using:

- /healthz for application availability.
- Prometheus for metrics collection.
- Grafana for dashboards and visualization.
- Filebeat for log collection.
- Logstash for log processing.
- Nginx access and error logs for request troubleshooting.
- AI-assisted log analysis for incident classification.

If a deployment fails, operators can inspect container status, logs, health checks, Kubernetes events, or AWS connectivity.

10. Final user output

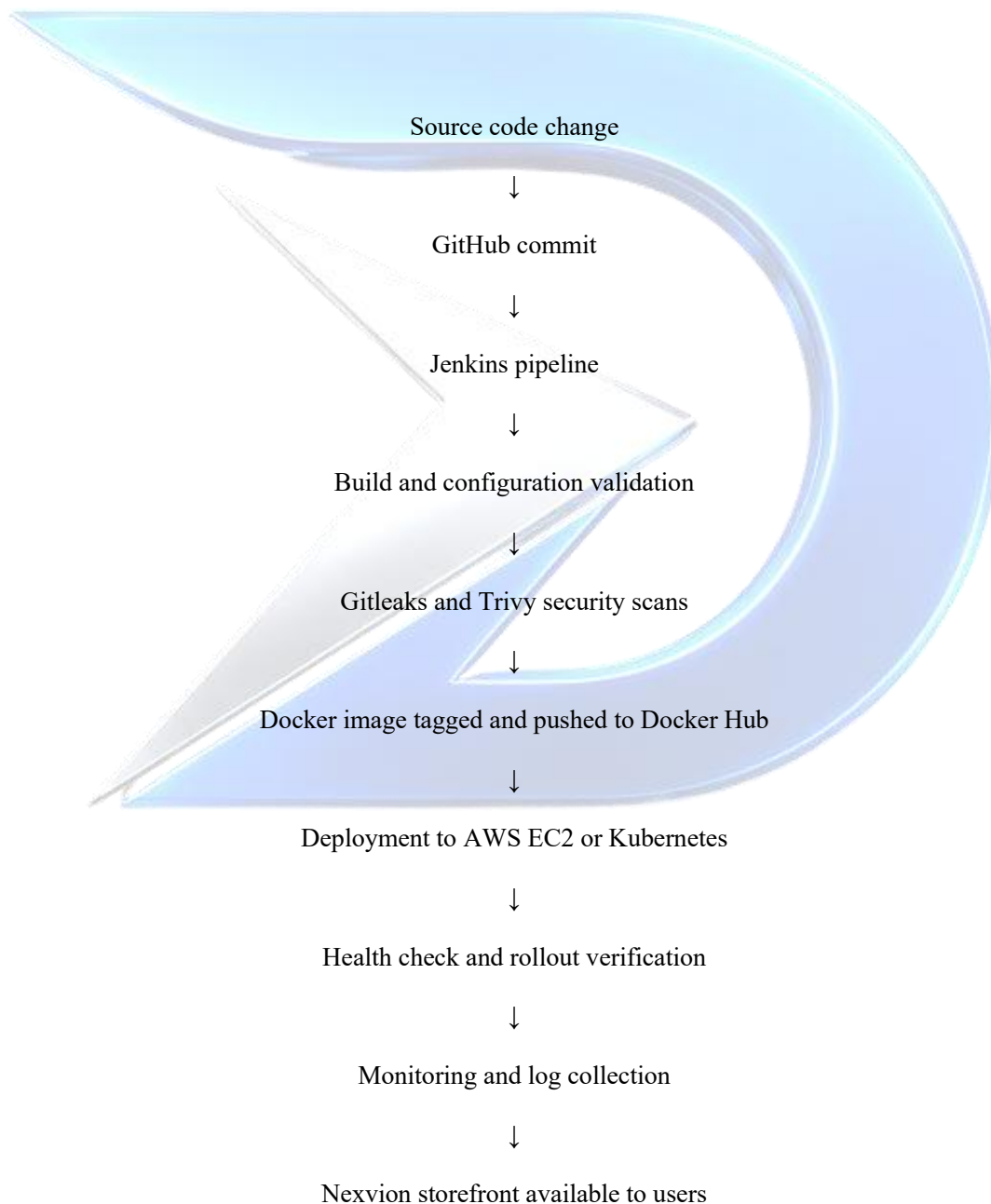
The end user accesses the deployed Nexvion storefront through a browser.

The user can:

- View the home page.
- Browse products.
- Search and filter products.
- Add products to a cart.
- Complete a demonstration checkout.
- Receive an order confirmation.

The final output is a working, accessible e-commerce storefront delivered through an automated and monitored DevOps pipeline.

Overall flow



8. Project Implementation

8.1 Environment & Initial Setup

The project was developed using Git, GitHub, Docker, Jenkins, Terraform, Ansible, AWS, Kubernetes, and Helm. The application was packaged as an unprivileged Nginx Docker image. AWS infrastructure was provisioned with Terraform, and the Ubuntu server was configured with Ansible, Docker, UFW, fail2ban, and SSH hardening.

8.2 Module / Feature Implementation

- **[Frontend]:** HTML, CSS, and JavaScript implement the home page, catalogue, search, filters, cart, login, and checkout.
- **Containerization:** Docker packages the static website with Nginx and provides the /healthz endpoint.
- **CI/CD:** Jenkins builds, validates, scans, publishes, deploys, and verifies the application.
- **Infrastructure:** Terraform creates the AWS network, EC2 instance, security group, encrypted storage, and Elastic IP.
- **Server configuration:** Ansible installs Docker, configures security, and deploys the production Compose service.
- **Kubernetes:** Manifests and Helm charts provide replicas, services, ingress, probes, and rolling updates.
- **Monitoring and logging:** Prometheus, Grafana, Filebeat, and Logstash provide operational visibility.

Screenshots may be included for the storefront, Jenkins pipeline, AWS deployment, Kubernetes resources, and monitoring dashboards.

8.3 Deployment / Execution

Locally, the application runs with Docker Compose and is accessed through <http://localhost:8088>. For cloud deployment, Terraform provisions AWS infrastructure, Ansible configures the EC2 server, and Jenkins deploys the Docker image. The public application is served through the AWS Elastic IP on port 80 and verified using the /healthz endpoint

9. Key Features / Functionalities

9.1 Feature List

1. **Responsive storefront** – Provides a user-friendly home page and product browsing interface.
2. **Product catalogue** – Displays available products with images, descriptions, prices, and stock information.
3. **Search and filtering** – Allows users to search products and filter them by category, price, and stock status.
4. **Product sorting** – Enables sorting by price, name, and other catalogue criteria.
5. **Shopping cart** – Allows users to add, remove, and adjust product quantities.
6. **Cart persistence** – Saves cart data in browser localStorage for continued use in the same browser.
7. **Demo user accounts** – Provides registration, login, and logout functionality for demonstration purposes.
8. **Checkout process** – Collects customer details and validates checkout information.
9. **Multiple payment demonstrations** – Supports card, UPI, and cash-on-delivery demo options.
10. **Order confirmation** – Displays confirmation details after a successful demo checkout.
11. **Containerized deployment** – Runs the storefront in a portable Docker image using Nginx.
12. **Health monitoring** – Provides a /healthz endpoint and container health checks.
13. **Automated CI/CD** – Jenkins builds, tests, scans, publishes, deploys, and verifies the application.
14. **Security validation** – Uses Gitleaks and Trivy to detect secrets and image/configuration vulnerabilities.
15. **Cloud deployment** – Runs on AWS EC2 with Terraform-provisioned networking and an Elastic IP.
16. **Server hardening** – Uses Ansible, UFW, fail2ban, SSH hardening, and non-root containers.
17. **Kubernetes support** – Provides deployments, services, ingress, probes, resource limits, and Helm packaging.
18. **Release strategy examples** – Includes rolling, blue/green, and canary deployment configurations.
19. **Monitoring and logging** – Provides Prometheus, Grafana, Filebeat, and Logstash configurations.
20. **Operational troubleshooting** – Includes documented procedures for diagnosing deployment, networking, container, and service failures.

10. Testing & Validation

10.1 Testing Approach

The project was tested at multiple levels:

- **Application testing:** Verified page navigation, product search, filtering, sorting, cart operations, login, checkout, and order confirmation.
- **Container testing:** Built the Docker image and validated the Nginx configuration.
- **Health testing:** Checked the /healthz endpoint locally and on AWS.
- **CI/CD testing:** Verified Jenkins build, security scanning, image publishing, deployment, and post-deployment checks.
- **Infrastructure testing:** Validated Terraform configuration and confirmed AWS networking, SSH, and HTTP access.
- **Security testing:** Used Gitleaks and Trivy to scan source code, images, and deployment configuration.
- **Deployment testing:** Verified Docker Compose, AWS EC2, and Kubernetes deployment configurations.

10.2 Test Cases

Test case	Expected result	Actual result	Status
Open home page	Page loads successfully	Page loaded successfully	Pass
Browse products	Product catalogue is displayed	Products displayed correctly	Pass
Search for a product	Matching products are shown	Search worked correctly	Pass
Filter and sort products	Results update according to selected options	Filtering and sorting worked	Pass
Add product to cart	Product appears in cart	Product added successfully	Pass
Change cart quantity	Cart total updates correctly	Quantity and total updated	Pass
Register and log in	Demo user can access the storefront	Demo authentication worked	Pass
Submit checkout form	Invalid data is rejected and valid data proceeds	Validation worked correctly	Pass
Complete demo order	Order confirmation is displayed	Confirmation displayed	Pass

Test case	Expected result	Actual result	Status
Build Docker image	Image builds without errors	Image built successfully	Pass
Validate Nginx configuration	Configuration test passes	Nginx validation passed	Pass
Check local health endpoint	/healthz returns 200 OK and ok	Health check passed	Pass
Check AWS health endpoint	Public endpoint responds successfully	AWS returned HTTP 200	Pass
SSH to AWS server	Authorized user can connect	SSH connection succeeded after CIDR correction	Pass
Run security scans	No blocking critical issue is accepted	Gitleaks and Trivy stages executed	Pass
Deploy container	Application starts and remains healthy	Container deployed successfully	Pass
Kubernetes readiness check	Pods become ready	Readiness and liveness probes configured	Pass

10.3 Validation

The final output was validated by:

- Opening the storefront through the browser.
- Testing the main user workflows from product browsing through checkout.
- Running the Docker container health check.
- Confirming the AWS public endpoint returned HTTP 200.
- Connecting to the EC2 instance through SSH.
- Inspecting the running Docker image and container status.
- Verifying Jenkins deployment and health-check stages.
- Reviewing Terraform, Ansible, Kubernetes, and Helm configurations.
- Running git diff --check to confirm documentation and configuration changes contained no whitespace errors.

The final application was confirmed to be deployable, accessible, health-checkable, and supported by documented operational procedures. Authentication and payment were validated only as demonstrations because they use browser localStorage rather than production backend services.

11. Results & Outcomes

11.1 Final Results

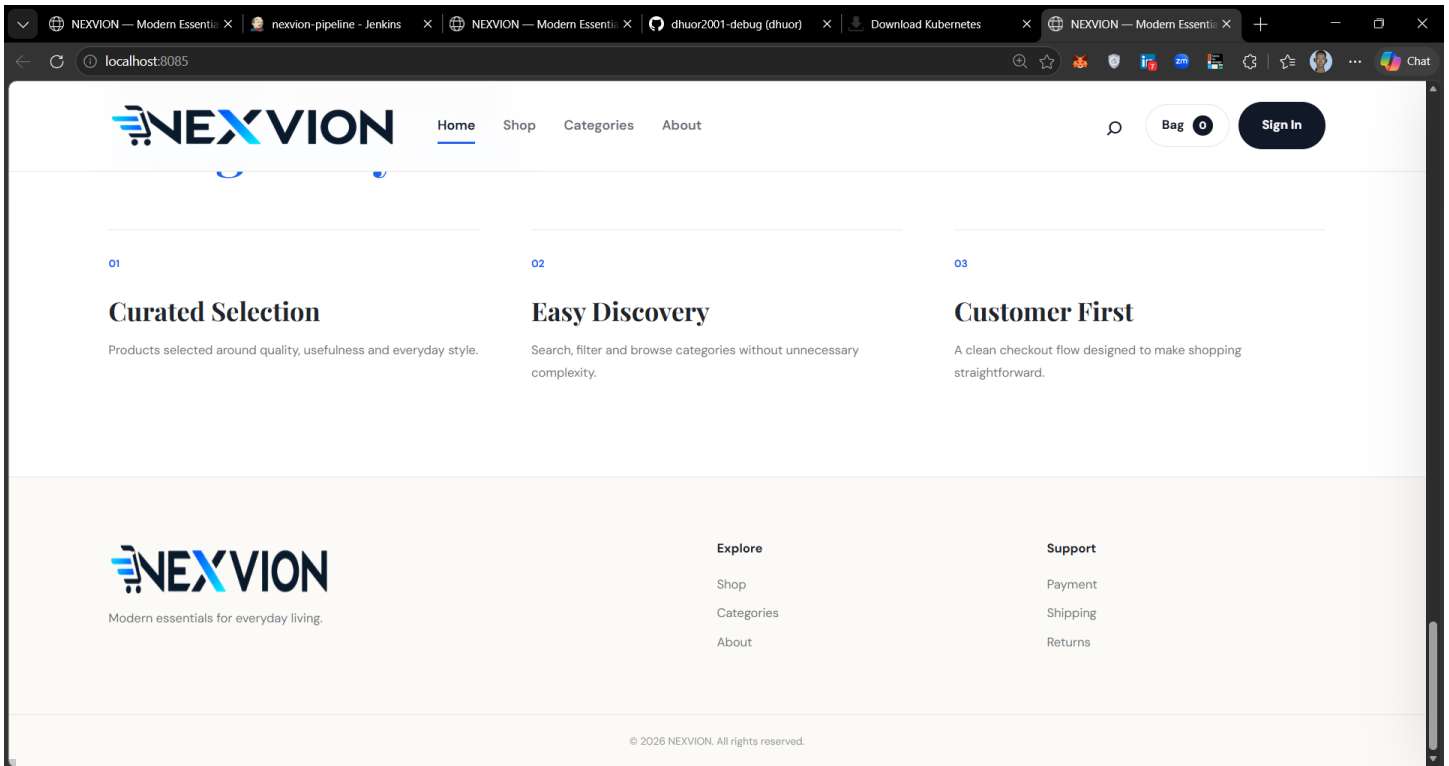
Nexvion was successfully developed as a containerized e-commerce storefront. It was built, security-scanned, deployed to AWS EC2, and verified using the /healthz endpoint. Docker, Jenkins, Terraform, Ansible, Kubernetes, monitoring, logging, and troubleshooting documentation were completed.

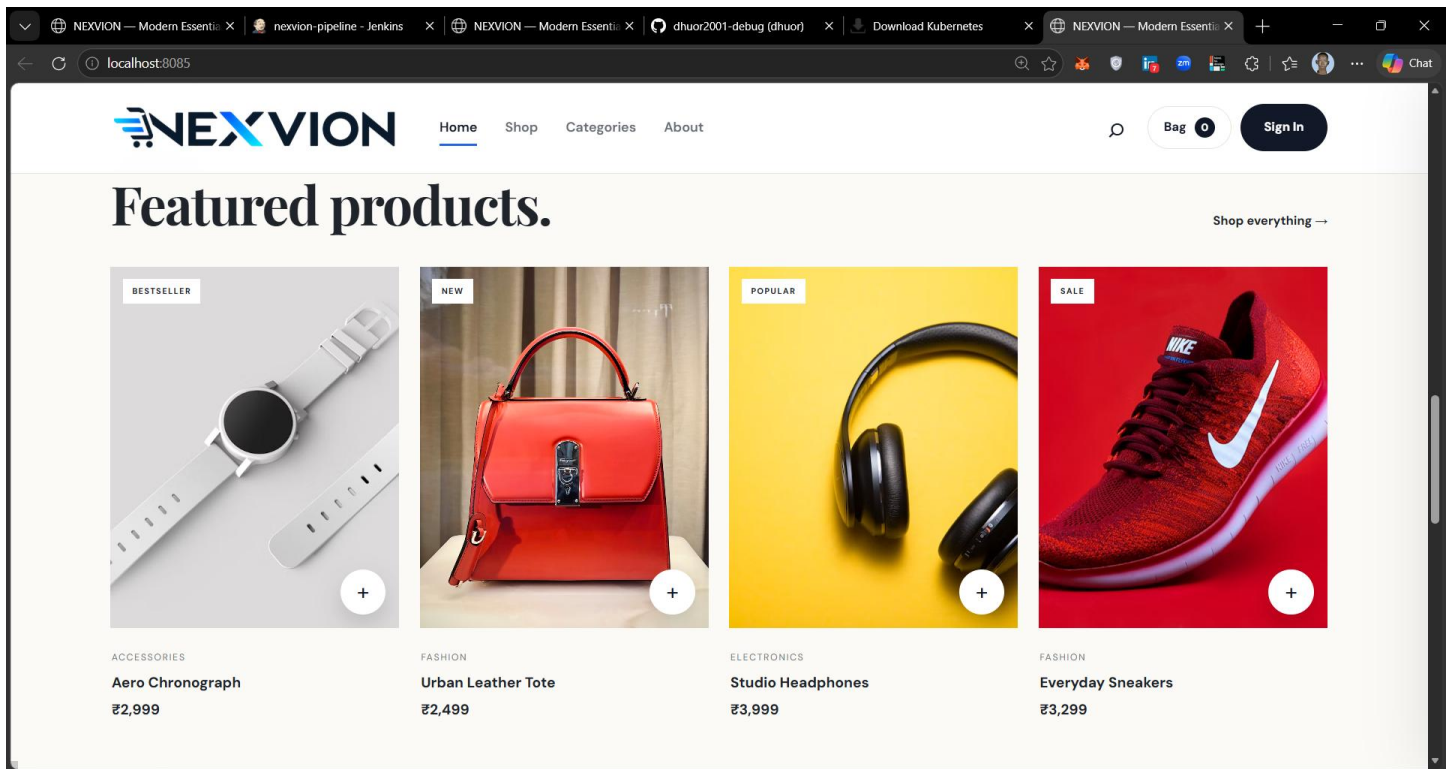
11.2 Outcome Evaluation

The main objectives were achieved. The project demonstrated a complete DevOps workflow from source code and automated testing to secure deployment, monitoring, and validation. However, authentication and payment remain demonstrations because no production backend or real payment gateway is included.

12. Project Screenshots & Evidence

12.1 Implementation Evidence





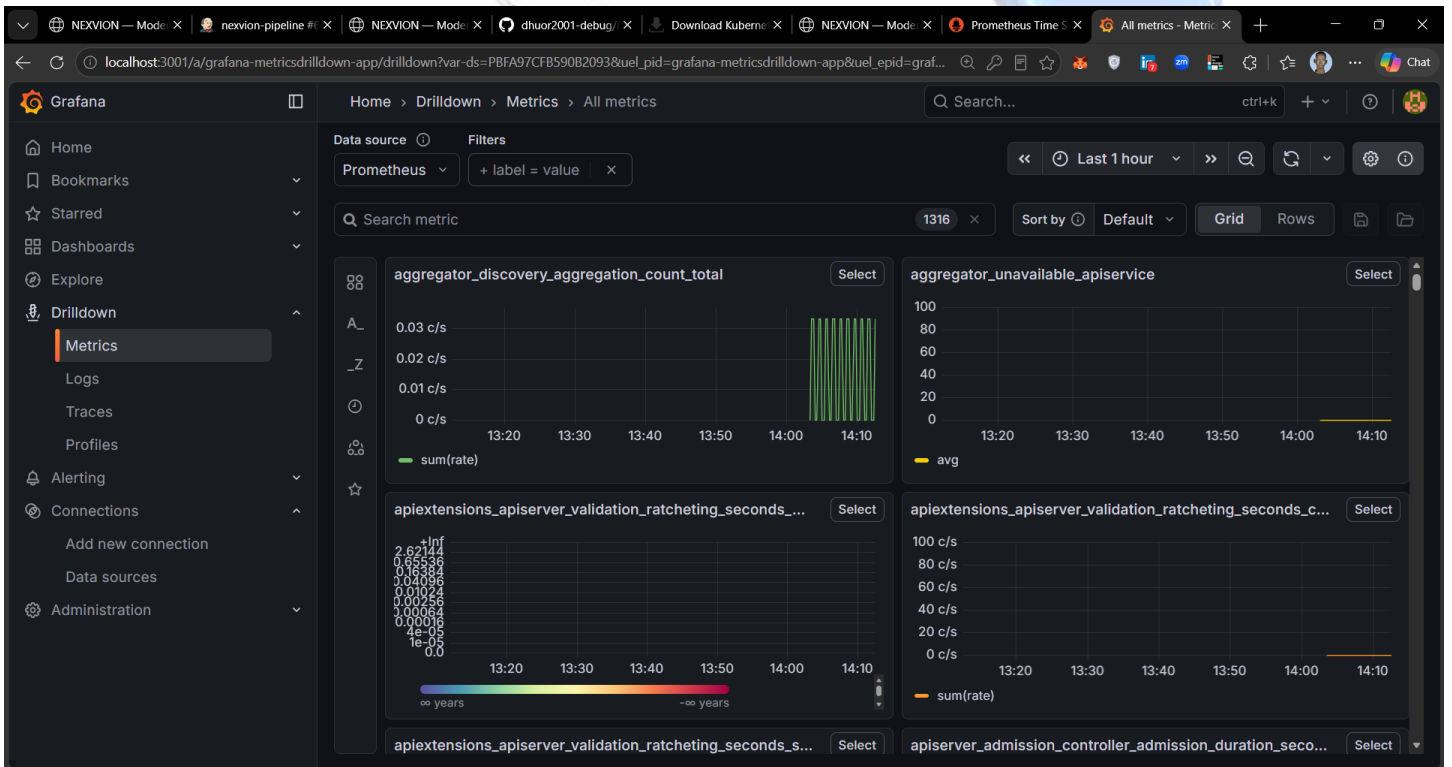
Personal access tokens

You can use a personal access token instead of a password for Docker CLI authentication. Create multiple tokens, control their scope, and delete tokens at any time. [Learn more](#)

[Generate new token](#)

Description	Status	Source	Created	Last used	Expiration date
jenkins-week9	Active	Manual	Sep 16, 2026 at 20:27:59	Sep 16, 2026 at 20:42:05	Never
dockerhub-creds	Active	Manual	Oct 06, 2026 at 22:24:14	Never	Never
jenkins-wee	Active	Manual	Sep 16, 2026 at 16:36:15	Never	Never

Rows per page: 10 1-3 of 3





13. Challenges & Solutions

13.1 Challenges Faced

- AWS SSH connection timed out because the allowed administrator IP was outdated.
- Jenkins, Docker, and application ports conflicted during local deployment.
- Container and Nginx upstream health checks initially required troubleshooting.
- Security scans detected test secrets and image/configuration risks.
- Kubernetes deployments required correct probes, secrets, ports, and ingress settings.

13.2 Solutions Applied

- Updated the Terraform `admin_cidr` to the current public IP.
- Used separate ports for Jenkins, the application, and the local proxy.
- Added and verified `/healthz`, container health checks, and correct port mappings.
- Used Gitleaks and Trivy, removed unsafe secrets, and applied container hardening.
- Configured Kubernetes probes, external secrets, resource limits, non-root execution, and ingress routing.



14. Learning Outcomes

14.1 Technical Learning

- Learned Docker containerization and Nginx configuration.
- Gained experience with Jenkins CI/CD pipelines.
- Learned Terraform-based AWS infrastructure provisioning.
- Practiced Ansible server automation and security hardening.
- Worked with Kubernetes, Helm, monitoring, logging, and security scanning.

14.2 Practical / Industry Learning

- Improved troubleshooting of real deployment and networking issues.
- Learned to automate repeatable build and deployment processes.
- Practiced secure handling of infrastructure, containers, and credentials.

- Gained experience documenting, validating, and operating a complete DevOps project.



15. Conclusion

15.1 Project Conclusion

Nexvion successfully achieved its main objectives by delivering a functional containerized storefront with automated CI/CD, security scanning, AWS deployment, infrastructure automation, and Kubernetes support. The project demonstrated a complete DevOps workflow from development to validated deployment.

Although authentication and payment remain demonstrations, the project provides a strong, practical foundation for learning, portfolio presentation, and future expansion into a fully production-ready e-commerce platform.



16. Future Enhancements

16.1 Possible Improvements

- Add a secure backend API, database, and server-side authentication.
- Integrate a real payment gateway and order-management system.
- Add inventory, shipping, tax, refund, and notification services.
- Enable HTTPS, domain management, automated TLS certificates, and stronger secret management.
- Replace the fixed AWS host with dynamic infrastructure outputs or AWS Systems Manager.
- Add automated unit, integration, browser, and load testing.
- Improve Kubernetes scalability with autoscaling, persistent storage, and high availability.
- Expand monitoring with alert rules, centralized dashboards, and log retention.
- Add backup, disaster recovery, rollback automation, and multi-region deployment.
- Improve accessibility, mobile responsiveness, performance, and image optimization.



17. References

17.1 Resources Used

- [Nexvion GitHub repository](#)
- [Docker Documentation](#)
- [Nginx Documentation](#)
- [Terraform Documentation](#)
- [AWS Documentation](#)
- [Ansible Documentation](#)

- [Jenkins Documentation](#)
- [Kubernetes Documentation](#)
- [Helm Documentation](#)
- [Prometheus Documentation](#)
- [Grafana Documentation](#)
- [Trivy Documentation](#)
- [Gitleaks Documentation](#)
- [Filebeat Documentation](#)
- [Logstash Documentation](#)
- [Ubuntu Server Documentation](#)
- [Git Documentation](#)

